Big Data y Machine Learning para Economía Aplicada

Week 07: Redes neuronales y clases desbalanceadas

Eduard F. Martinez Gonzalez

Departamento de Economía, Universidad Icesi

October 27, 2026

Previously...

- **Abrimos la caja del boosting:** el departamento primero, la etnia después; los cruces que encontró la máquina son, sobre todo, la etnia en cada departamento (afro $\times$ departamento: 2,4 puntos).
- **La pista:** permutar un bloque del censo subía el error entre 1,3 y 3,8 puntos; reentrenar sin él, sólo entre 0,3 y 0,4. Los bloques se reemplazan entre sí: las 37 columnas cuentan pocas historias, repetidas.
- **Y dónde falla:** 14,9 de error en lo rural contra 9,3 en las cabeceras. Con el mapa, otra explicación con casi el mismo error (12,1).

Lo que queda en el kit

Un modelo por abrir: la **red neuronal**. Y una deuda de clasificación que arrastramos desde la semana 4: ¿qué se hace cuando una clase casi no aparece en los datos?

Debrief del Problem Set 1: el ranking

Cinco entregas, ordenadas por el RMSE de prueba del precio (millones de pesos)

	Equipo	Modelo final	Validación RMSE	Validación esquema	Prueba RMSE
1	Andrés Martínez y Lina Vélez	tres modelos combinados y el precio del *gemelo*	159,1	azar	**165,2**
2	Laura Sinisterra	bosque aleatorio sobre log(precio), con el censo por manzana	164,8	azar	174,2
3	Elkin Mola y Danilo Vanoy	regresión de log(precio) con estrato y distancias	181,8	zonas	181,8
4	Juan Felipe Vargas	Lasso: la ubicación multiplica área, tipo y baños	196,4	zonas	197,7
5	Iván Villalba	regresión con comuna, coordenadas e interacciones	179,4	80/20	204,4
	Referencia: M2, la regresión de la Parte 1 (área, baños, tipo y coordenadas)				225,3
	Referencia: M0, la media del entrenamiento				355,2

- **Cómo leerla:** RMSE en los 3.819 avisos de prueba. *Validación*: lo que cada equipo prometió con su esquema; *prueba*: lo que dio. Los cinco le ganan a M2.
- **¿Coinciden?** En cuatro de cinco, a menos de 10 millones (por zonas, a 0 y 1; al azar, 6 y 9 por debajo de la prueba). La quinta prometió 179 y dio 204.

La pregunta de hoy: el último modelo del curso, en clasificación

Dos temas: una deuda (las clases desbalanceadas) y la red neuronal, pieza por pieza

- **La deuda primero:** desde la semana 4 clasificamos puestos sin preguntar qué pasa cuando una clase es **rara**. Hoy la pagamos con la **primera vuelta de 2022**: cuatro candidatos; uno gana el 54,2 % de los puestos y otro, el 0,1 %.
- **Después, la red neuronal:** el último modelo del kit, armado pieza por pieza: la unidad, la activación, la capa y la salida. Primero para la pregunta de siempre —¿gana Petro?— y después con **cuatro clases a la vez**: ¿Petro, Hernández, Fico u otro?
- **El cierre de siempre:** la carrera contra la logística (semana 4) y los árboles (semana 5). No tiene que ganar: hoy el punto es entender cómo funciona.

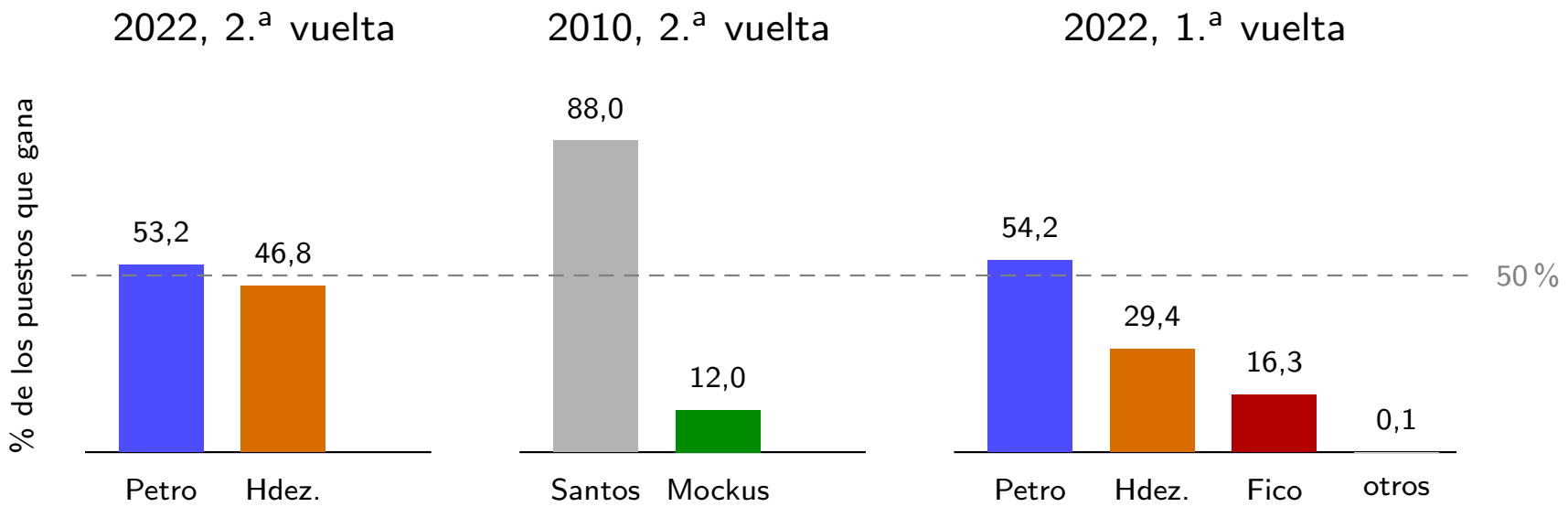
La idea con la que se van

Una red neuronal es la logística de la semana 4 con variables que ella misma fabrica. Y con una clase rara, la exactitud engaña: se decide con la métrica y el umbral, no inflando los datos.

1. La deuda: clases desbalanceadas
2. La red neuronal, pieza por pieza
3. Cómo se aprende una red
4. El ejemplo binario: ¿gana Petro el puesto?
5. Más de dos categorías: la primera vuelta
6. La carrera y el cierre

La *y* de hoy: tres elecciones, tres repartos

El % de los puestos que gana cada candidato; ganar es sacar más votos que cualquier otro



- **Cómo leerla:** cada barra es el porcentaje de puestos que gana un candidato (2022 en el entrenamiento; 2010, toda la tabla); la línea punteada marca el 50 %. A la derecha entra hoy la **primera vuelta de 2022**: 11.948 de nuestros 12.000 puestos (el resto no empata con la segunda vuelta o quedó empatado).
- **El reparto importa:** en la segunda vuelta de 2022 las clases están casi parejas (53/47). En 2010, Santos ganó el 88,0 % de los puestos. Y la primera vuelta tiene **cuatro clases**: Petro 54,2 %, Hernández 29,4 %, Fico 16,3 % y *otros*, 0,1 %.

Recap (I): la matriz de confusión

Antes de medir con una clase rara: cada puesto de prueba cae en una de cuatro celdas

	el modelo dice "Fico"	**dice "no Fico"**	total
Fico gana	VP $= 204$	FN $= 199$	403
no gana	FP $= 69$	VN $= 1.916$	1.985
total	273	2.115	$n = 2.388$

- **Cómo leerla:** la logística de la semana 4 en los 2.388 puestos de prueba de la primera vuelta, decidiendo con $\tau = 0{,}5$. Las **filas** son la verdad; las **columnas**, lo que dice el modelo. En azul, los aciertos; en rojo, los dos errores.
- **VP** (verdaderos positivos): Fico gana y el modelo lo dice. **FN** (falsos negativos): Fico gana y el modelo no lo ve. **FP** (falsos positivos): el modelo dice Fico y no ganó. **VN**: ni gana ni lo dice.
- **Los totales:** las filas suman lo que pasó (403 puestos de Fico, 1.985 sin Fico); las columnas, lo que el modelo señaló (273 y 2.115). Las medidas de la lámina siguiente sólo cambian de denominador.

Recap (II): cuatro medidas, cuatro denominadores

La misma matriz, leída por todos, por la fila de Fico o por la columna de lo señalado

Medida	Fórmula y cuenta	Qué pregunta
Exactitud	$\frac{VP+VN}{n} = \frac{2.120}{2.388} = 88{,}8\,\%$	¿qué fracción de *todos* clasifica bien?
Sensibilidad (*recall*)	$\frac{VP}{VP+FN} = \frac{204}{403} = 50{,}6\,\%$	de los puestos donde Fico *gana* (la fila), ¿cuántos encuentra?
Precisión	$\frac{VP}{VP+FP} = \frac{204}{273} = 74{,}7\,\%$	de los que *señala* (la columna), ¿en cuántos gana Fico?
Especificidad	$\frac{VN}{VN+FP} = \frac{1.916}{1.985} = 96{,}5\,\%$	de los puestos donde *no* gana, ¿cuántos descarta?

Por qué pueden contar historias distintas

Cada una divide por otra cosa. La exactitud, por todos: cuando una clase es rara la llenan los VN, y decir "nunca" ya acierta mucho. La sensibilidad mira sólo la fila de Fico; la precisión, la columna de lo señalado. Señalar más puestos sube la sensibilidad y suele bajar la precisión.

Por qué la exactitud engaña cuando una clase es rara

¿Gana Fico el puesto? La clase aparece en el 16,3 % del entrenamiento

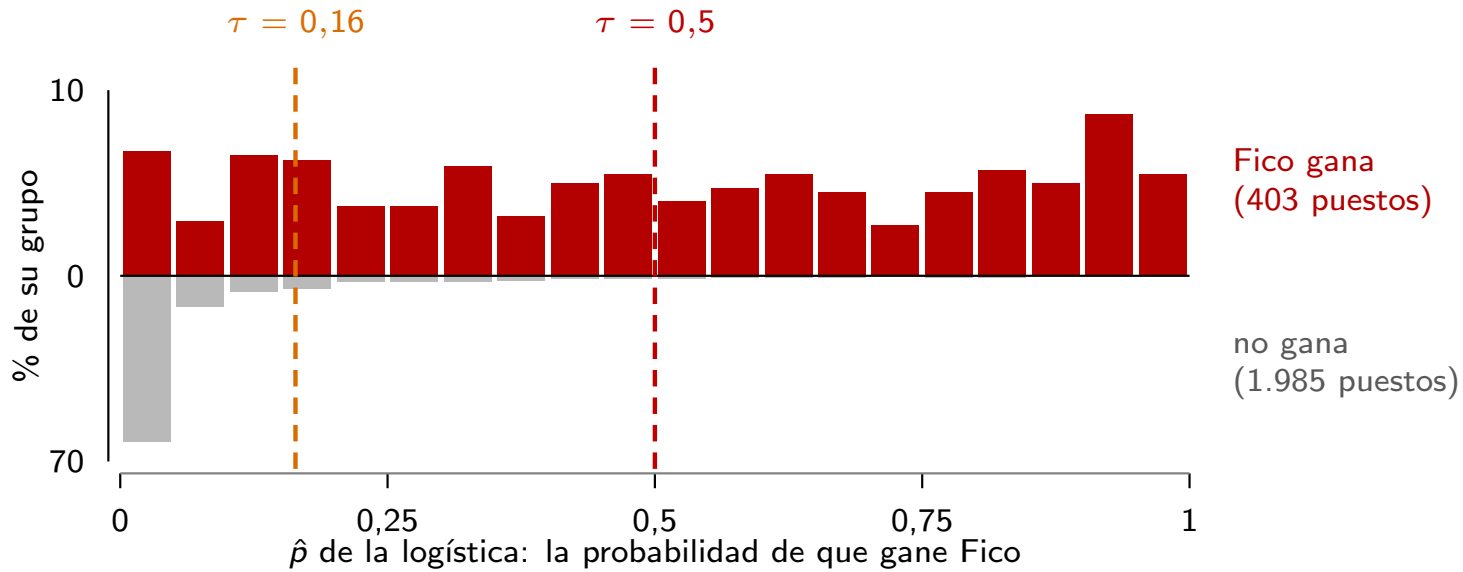
La referencia de la semana 4 —decir siempre la clase mayoritaria— aquí es "Fico nunca gana": acierta el **83,1 %** de la prueba sin mirar un solo dato. **La logística de la semana 4** (censo + *dummies* de departamento), con el umbral de siempre:

	Exactitud (%)	Sensibilidad (%)	Precisión (%)
"Fico nunca gana"	83,1	0,0	—
Logística, $\tau = 0{,}5$	88,8	50,6	74,7

- **La exactitud suena espléndida** (88,8 %)... pero la referencia que no modela nada ya acertaba 83,1: con clases desbalanceadas, el listón de la exactitud está regalado.
- **La sensibilidad dice la verdad incómoda:** de cada 100 puestos donde Fico gana, la regla de $\tau = 0{,}5$ encuentra 50,6.
- **El problema no es el modelo:** ordena bien (AUC 0,902). Es la **regla de decisión**: con una clase rara, exigir $\hat{p} \geq 0{,}5$ es exigir demasiado.

La primera salida (I): qué decide el umbral τ

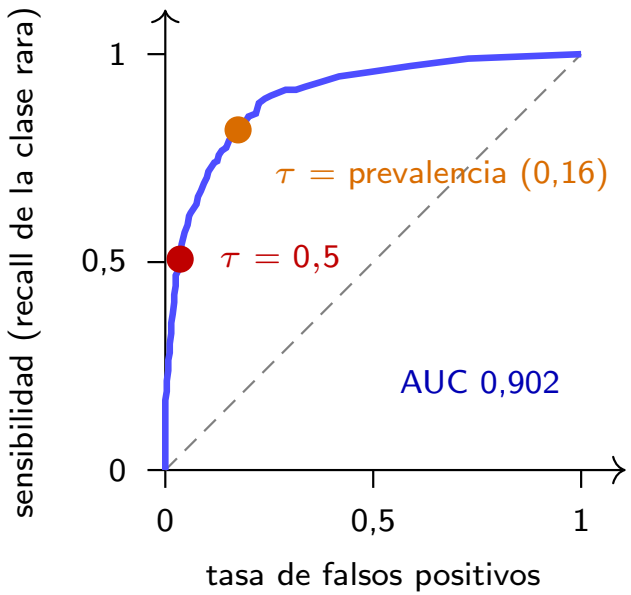
El modelo entrega una probabilidad; el umbral la convierte en una decisión



- **Cómo leerla:** arriba, los 403 puestos de prueba donde Fico gana; abajo, los 1.985 donde no gana (cada mitad con su escala). Cada barra es el % de su grupo con esa $\hat{p}$. Lo que queda a la **derecha** de una línea, el modelo lo señala como "Fico".
- **Con** $\tau = 0{,}5$ (línea roja) pasa la mitad de los rojos: sensibilidad 50,6 % (204 encontrados, 199 perdidos). De los grises pasa el 3,5 %: 69 falsas alarmas.
- **Con** $\tau = 0{,}16$ (naranja) pasan el 81,9 % de los rojos (330) y el 17,6 % de los grises (350): la precisión cae a 48,5 %. **El modelo es el mismo; cambió la regla.**

La primera salida (II): la ROC, todos los umbrales a la vez

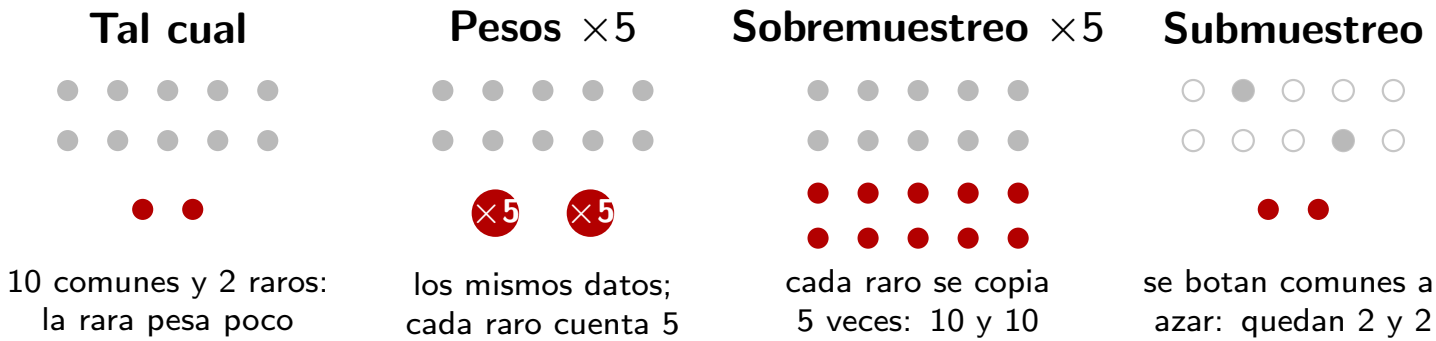
Cada punto de la curva es un umbral; bajarlo sube el punto y lo corre a la derecha



- **Cómo leerla:** el eje vertical es la sensibilidad (los rojos a la derecha de τ en la lámina anterior); el horizontal, la tasa de falsos positivos, $FP/(FP + VN) = 1-$ especificidad (los grises a la derecha). Barrer τ de 1 a 0 dibuja la curva azul.
- **Los dos puntos son el mismo modelo:** $\tau = 0{,}5$ (rojo: 50,6 % y 3,5 %) y $\tau = 0{,}16$ (naranja: 81,9 % y 17,6 %). Mover τ no cambia la curva ni la AUC (0,902): cambia el punto donde decidimos, y cuál conviene lo deciden los **costos** (semana 4).

La segunda salida (I): tocar los datos o la pérdida

Tres maneras de que el modelo "vea" más casos raros; ninguna agrega información



- **Pesos:** los mismos datos, pero en la log-loss de la semana 4 cada puesto raro cuenta por cinco: $-\sum_i w_i \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right]$, con $w_i = 5$ si Fico gana y $w_i = 1$ si no.
- **Sobremuestreo** (*oversampling*): copiar cada fila rara cinco veces; cada copia suma el mismo término, así que es *la misma* pérdida que con pesos. **Submuestreo** (*undersampling*): botar puestos comunes al azar hasta igualar a los raros.
- **Las tres buscan lo mismo:** que la clase rara pese más al estimar. Y pagan lo mismo: el modelo cree que la clase es más frecuente de lo que es. Se hacen sólo con el entrenamiento, dentro de cada pliegue; la prueba se queda como es.

La segunda salida (II): lo que pasa en el caso

La logística de Fico con cada salida, decidiendo siempre con $\tau = 0{,}5$

	AUC	Exact. (%)	Sensib. (%)	Precis. (%)	$\hat{p}$ media (%)
Tal cual, $\tau = 0{,}5$	0,902	88,8	50,6	74,7	16,3
Tal cual, $\tau = 0{,}16$	0,902	82,3	81,9	48,5	16,3
Pesos $\times 5$ a la rara	0,904	83,0	82,4	49,7	31,8
Sobremuestreo $\times 5$	0,904	83,0	82,4	49,7	31,8
Submuestreo	0,904	82,5	82,6	48,8	32,2

- **La AUC casi no se mueve** (0,902 $\rightarrow$ 0,904): ponderar o remuestrear no crea información; el modelo ordena los puestos igual. Y mire las filas de pesos y sobremuestreo: **idénticas**, como anunciaba la fórmula: ponderar por 5 *es* repetir cada puesto raro 5 veces. Las tres mueven el punto de decisión, que es lo que ya hacía el umbral, por la puerta de atrás.
- **El precio escondido:** la $\hat{p}$ media sube del 16,3 % (lo que de verdad ocurre: 16,9 %) al 31,8 %: las probabilidades **dejan de ser probabilidades**. Si remuestrea, recalibre después, y siempre **dentro del pliegue** (regla de oro #3).

Las reglas del desbalance

Lo que debimos decir en la semana 4; desde hoy, parte del kit

1. **Mire el reparto de la *y* antes que nada** (describir el *target*: la regla de la semana 2). Con una clase rara, la exactitud no es la métrica: AUC, sensibilidad y precisión de la clase rara, o la exactitud balanceada (semana 4).
2. **La primera palanca es el umbral,** y lo fijan los costos (semana 4). Es gratis, no toca los datos y no daña las probabilidades.
3. **Ponderar o remuestrear equivale a mover el umbral:** no crea información y descalibra. Si lo hace, recalibre después y hágalo **dentro del pliegue** (regla de oro #3).
4. **Hay clases que no se pueden aprender:** *otros* gana el 0,1 % de los puestos; con tan pocos casos no hay pliegue que alcance. Se agrupa con otra clase o se cambia la pregunta.

1 La deuda: clases desbalanceadas

2 La red neuronal, pieza por pieza

3 Cómo se aprende una red

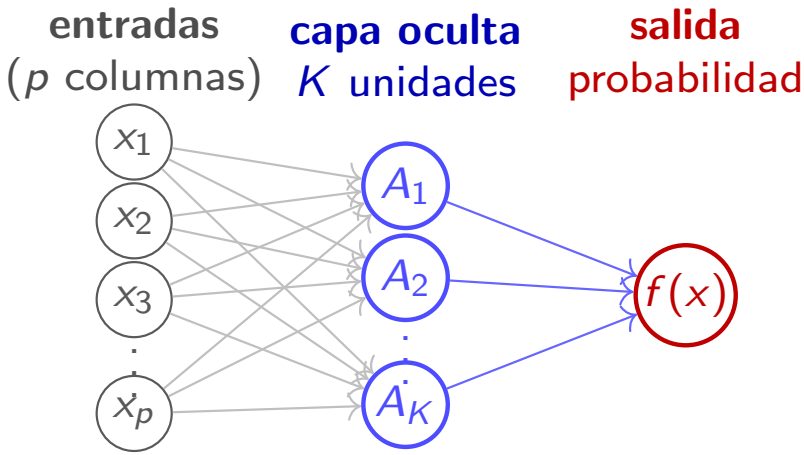
4 El ejemplo binario: ¿gana Petro el puesto?

5 Más de dos categorías: la primera vuelta

6 La carrera y el cierre

La red neuronal, de lejos

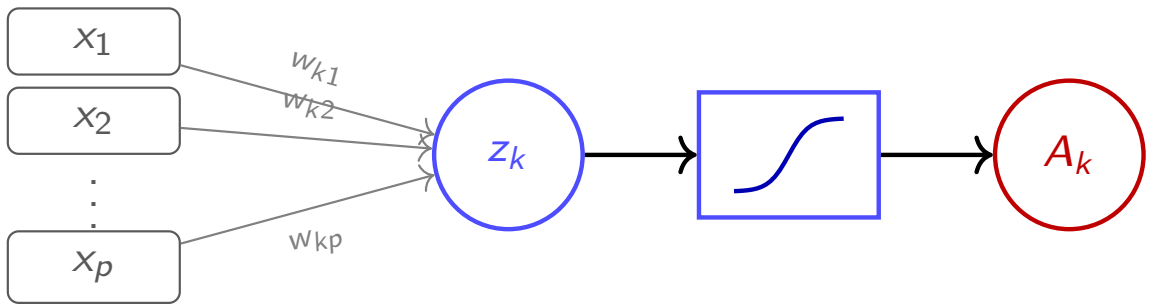
ISL 10.1: un modelo en dos etapas que fabrica sus propias variables. Hoy lo armamos pieza por pieza



- **Dos etapas:** la capa oculta (*hidden layer*) vuelve las entradas (censo + departamento: 70 columnas) K variables nuevas, $A_1, \ldots, A_K$ (las **activaciones**); la salida las vuelve una **probabilidad**: la de que Petro gane. Todo se aprende junto, mirando la y.
- **Hoy, todo en clasificación** (la y de la semana 4); el mismo esqueleto hace regresión cambiando la salida. Al final, cuatro probabilidades a la vez: la primera vuelta.
- **Las piezas, en orden:** la **unidad** (*hidden unit*: qué fabrica), la **capa y la salida** (cuántas entran y salen) y la **activación** g (por qué se dobla). Después, cómo se aprende.

La pieza 1 (I): la unidad oculta, paso a paso

Cada unidad fabrica una variable nueva: un índice del censo, elegido por la y



$$\underbrace{z_k = w_{k0} + w_{k1}x_1 + \cdots + w_{kp}x_p}_{\text{1. una suma ponderada: un índice}} \qquad \underbrace{A_k = g(z_k)}_{\text{2. el índice, doblado por la curva } g}$$

- **Paso 1:** la suma es el predictor lineal de una regresión: los 70 datos del puesto se vuelven un solo número, z_k. Los pesos $w_{k0}, \ldots, w_{kp}$ ($p+1$ por unidad) no se los damos: se aprenden para predecir la y.
- **Paso 2:** la curva g convierte ese número en la **activación** A_k. Con la sigmoide de la semana 4, A_k queda entre 0 y 1: qué tanto se *enciende* la unidad en ese puesto. Por qué hace falta doblar: la pieza 3.

La pieza 1 (II): una variable que nadie escribió

Un número por puesto, como una columna más de la base, pero fabricada por la red

Las cuatro unidades de la red que vamos a elegir ($K = 4$), leídas en los puestos de prueba: a qué variable del censo se parece cada activación.

Unidad	Se mueve con…	Correlación	Encendida ($A_k > 0{,}5$)
1	la población afro	$-0{,}43$	55 % de los puestos
2	el internet	$-0{,}75$	59 %
3	el gas domiciliario	$-0{,}64$	53 %
4	la población de 45 a 59 años	0,38	23 %

- **Una lectura, no una definición:** cada unidad mezcla las 70 entradas, así que ninguna *es* una variable del censo; la correlación sólo dice a cuál se parece. Una correlación negativa: la unidad se apaga donde esa variable es alta.

Pregunta para la sala: una red *sin* capa oculta —las entradas directo a una salida sigmoide—, ¿qué modelo es?

La pieza 1 (II): una variable que nadie escribió

Un número por puesto, como una columna más de la base, pero fabricada por la red

Las cuatro unidades de la red que vamos a elegir ($K = 4$), leídas en los puestos de prueba: a qué variable del censo se parece cada activación.

Unidad	Se mueve con…	Correlación	Encendida ($A_k > 0{,}5$)
1	la población afro	−0,43	55 % de los puestos
2	el internet	−0,75	59 %
3	el gas domiciliario	−0,64	53 %
4	la población de 45 a 59 años	0,38	23 %

- **Una lectura, no una definición:** cada unidad mezcla las 70 entradas, así que ninguna *es* una variable del censo; la correlación sólo dice a cuál se parece. Una correlación negativa: la unidad se apaga donde esa variable es alta.

Pregunta para la sala: una red *sin* capa oculta —las entradas directo a una salida sigmoide—, ¿qué modelo es?

La regresión logística de la semana 4: la red neuronal más pequeña del curso. Lo que la red agrega son las unidades: índices aprendidos *antes* de la sigmoide.

La pieza 2: cuántas entran, cuántas salen

K unidades en paralelo fabrican *K* variables; la salida las convierte en probabilidad

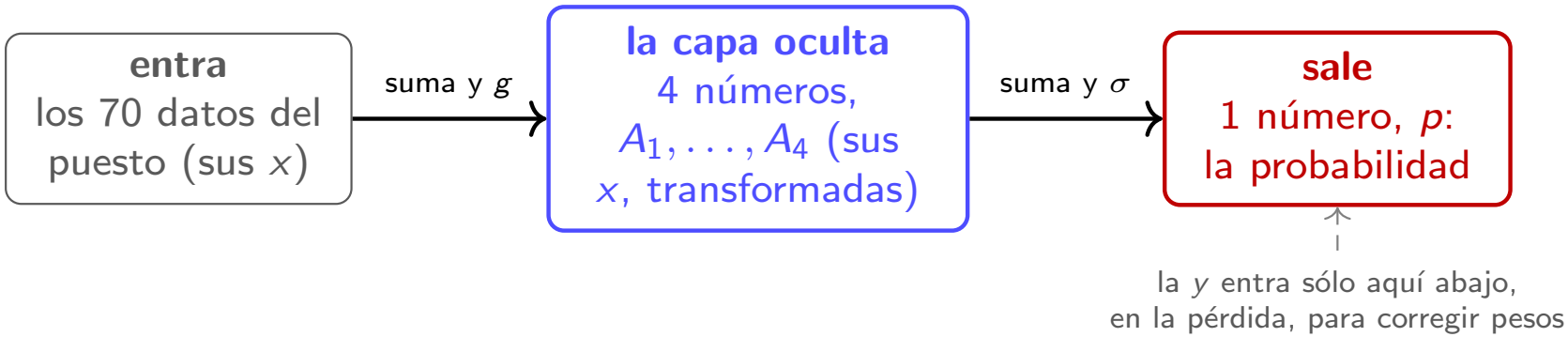
La capa oculta repite la unidad K veces, cada una con sus propios pesos. **La salida** es una logística, pero sobre las K activaciones, no sobre las x:

$$p(x) \;=\; \sigma\big(\beta_0 + \beta_1 A_1 + \cdots + \beta_K A_K\big), \qquad \sigma(t) = 1/(1+e^{-t}).$$

- **Entran p, sale 1:** entran las 70 columnas (censo + *dummies*) y sale **una** probabilidad, como en la logística. Para **regresión**, la misma red con la salida sin sigmoide; para **varias clases**, varias probabilidades (parte 5).
- **Cuántos parámetros:** cada unidad tiene $p+1$ pesos y la salida, $K+1$. Con 70 columnas y $K=4$: $4\times 71+5 = 289$; con $K=16$, 1.153. K es la **perilla de capacidad** de la red.
- **¿Y más capas?** Una segunda capa haría con las A lo mismo que la primera hizo con las x: sumas ponderadas, dobladas por g: índices *de los índices*. Hoy basta una, la que ajusta `nnet`.

Lo que viaja por la red: dos puestos, de punta a punta

La red elegida ($K = 4$): cada capa lee lo que fabricó la anterior; la y no viaja



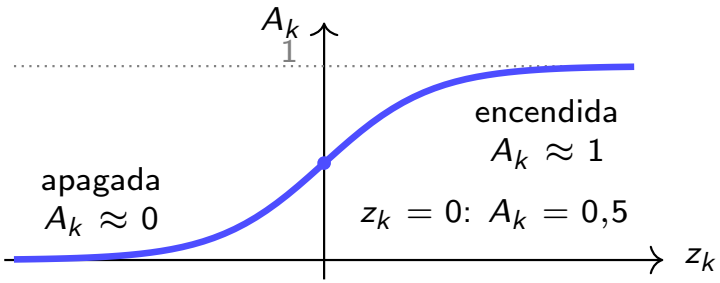
	A_1	A_2	A_3	A_4	p	¿Ganó Petro?
Pedro Grau Arola, Quibdó	0,09	0,79	0,95	0,01	0,99	sí (88,8 %)
Julia Agudelo, Medellín	0,78	0,04	0,02	0,59	0,16	no (40,2 %)
Pesos de la salida, β_k	−6,27	−11,59	6,72	−8,40	$\beta_0 = 8{,}60$	

- **Lo que pasa de capa en capa son las x del puesto, transformadas.** La capa oculta lee los 70 datos y entrega cuatro números, ya doblados por la curva. La salida lee esos cuatro: en Quibdó, $t = \beta_0 + \beta_1 A_1 + \cdots + \beta_4 A_4 = 5{,}28$ y $p = \sigma(t) = 0{,}99$; en Medellín, $t = -1{,}64$ y $p = 0{,}16$.
- **Lo que no pasa: la y.** Ni la y ni un residuo viajan hacia adelante; en el *boosting*, en cambio, cada árbol leía el residuo del anterior (semana 5). La y aparece una sola vez, al final, en la pérdida, para corregir los pesos: parte 3.

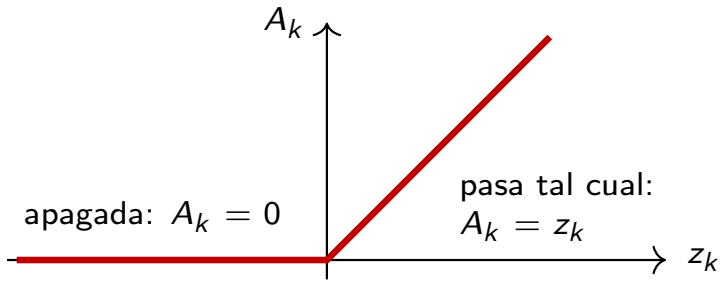
La pieza 3 (I): la activación g, un interruptor

g recibe el índice z_k de la unidad y decide cuánto se enciende en ese puesto

la sigmoide (la que usa `nnet`)



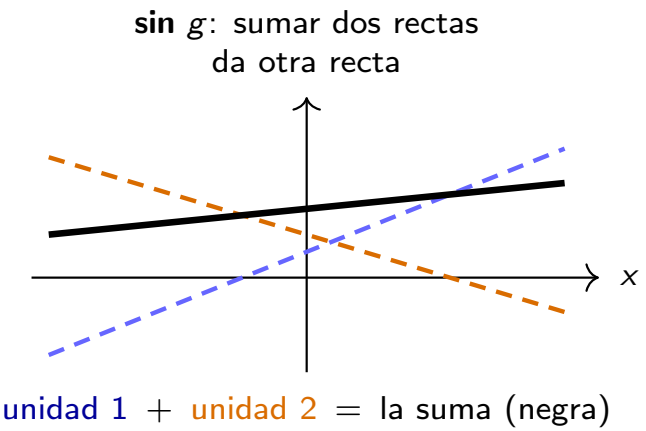
ReLU, $\max(0, z_k)$

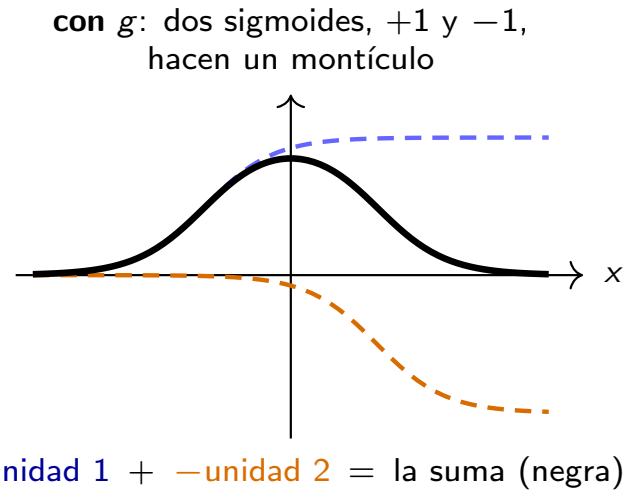


- **La sigmoide** (la de la semana 4): $g(z) = 1/(1 + e^{-z})$. Con z_k muy negativo la unidad se **apaga** ($A_k \approx 0$); con z_k muy positivo se **enciende** ($A_k \approx 1$); en $z_k = 0$ vale 0,5. Un interruptor suave.
- **ReLU**, $g(z) = \max(0, z)$: apaga lo negativo y deja pasar lo positivo tal cual. Un codo. Es la del *deep learning*, porque es barata de calcular.
- **Con los dos puestos de la lámina anterior:** los mismos pesos dan, en la unidad 1, $z_1 = -2{,}37$ en Quibdó, que la apaga ($A_1 = 0{,}09$), y $z_1 = 1{,}28$ en Medellín, que la enciende ($A_1 = 0{,}78$).

La pieza 3 (II): por qué la curva no puede faltar

ISL 10.1: sin g, sumar unidades da otra recta; con g, aparecen las formas y los cruces





- **Sin** g (o con $g(z) = z$) la red colapsa: $\sigma\left(\beta_0 + \sum_k \beta_k(w_{k0} + w_k' x)\right)$ es una sigmoide sobre otra recta en x: **la logística de la semana 4**, por más unidades que apile (izquierda).
- **Con** g, dos unidades ya hacen un *montículo* que ninguna recta puede hacer (derecha); con más unidades, más montículos: cualquier forma suave (apéndice A3). Y los cruces: con $g(z) = z^2$, $\frac{1}{4}\left[(x_1 + x_2)^2 - (x_1 - x_2)^2\right] = x_1 x_2$ (ISL 10.6 a 10.8), lo que los árboles de la semana 5 hacían con un corte dentro de otro.
- **El precio:** con muchas unidades, la red también aprende el ruido. ¿Cuántas? Con validación cruzada, en la parte 4.

1 La deuda: clases desbalanceadas

2 La red neuronal, pieza por pieza

3 Cómo se aprende una red

4 El ejemplo binario: ¿gana Petro el puesto?

5 Más de dos categorías: la primera vuelta

6 La carrera y el cierre

La pérdida: la verosimilitud de la semana 4, y bajar por el gradiente

ISL 10.7

El criterio es conocido: elegir los pesos $\theta = (\beta, w_1, \ldots, w_K)$ que minimizan la **log-loss** de la semana 4,

$$R(\theta) \;=\; -\sum_{i=1}^{n} \Big[y_i \log p_\theta(x_i) + (1 - y_i) \log \big(1 - p_\theta(x_i)\big) \Big]$$

(para regresión sería el RSS). **Lo nuevo:** por el anidamiento de los pesos y la simetría entre unidades, R **no es convexa**: tiene muchos mínimos locales. Adiós a las garantías de la logística.

El descenso de gradiente (*gradient descent*): arrancar en pesos al azar θ^0 y dar pasos cuesta abajo,

$$\theta^{t+1} \;=\; \theta^t - \rho \nabla R(\theta^t),$$

con ρ el **learning rate** (la tasa de aprendizaje). En redes grandes, el gradiente se calcula por *mini-batches* (*stochastic gradient descent*, SGD; apéndice A2).

- **Dos frenos contra el sobreajuste** (ISL): penalizar los pesos (*weight decay* = Ridge, semana 3) y parar a tiempo (*early stopping*; las rondas del *boosting*, semana 5): parte 4.

Backpropagation: el residuo, repartido

ISL 10.7.1; la derivación completa, en el apéndice A1

Con la log-loss y la salida sigmoide, el gradiente de una observación **empieza por el residuo** $y_i - p(x_i)$: la condición de primer orden de la logística, otra vez (semana 4):

$$\frac{\partial R_i}{\partial \beta_k} = -\big(y_i - p(x_i)\big)\, A_{ik}, \qquad \frac{\partial R_i}{\partial w_{kj}} = -\big(y_i - p(x_i)\big)\, \beta_k\, g'(z_{ik})\, x_{ij}.$$

- **Derivar reparte el residuo** entre los pesos: primero entre las unidades, según cuánto se activaron; después entre las entradas, a través de cada unidad. De ahí el nombre: el error se propaga de la salida hacia atrás.
- Es el residuo del *boosting* otra vez (semana 5): allí cada árbol nuevo ajustaba $y - \hat{p}$; aquí cada paso mueve cada peso en proporción a su parte de ese residuo.
- Nadie lo deriva a mano: los paquetes lo hacen solos (*automatic differentiation*). El costo del gradiente completo es una pasada hacia adelante y una hacia atrás.

1 La deuda: clases desbalanceadas

2 La red neuronal, pieza por pieza

3 Cómo se aprende una red

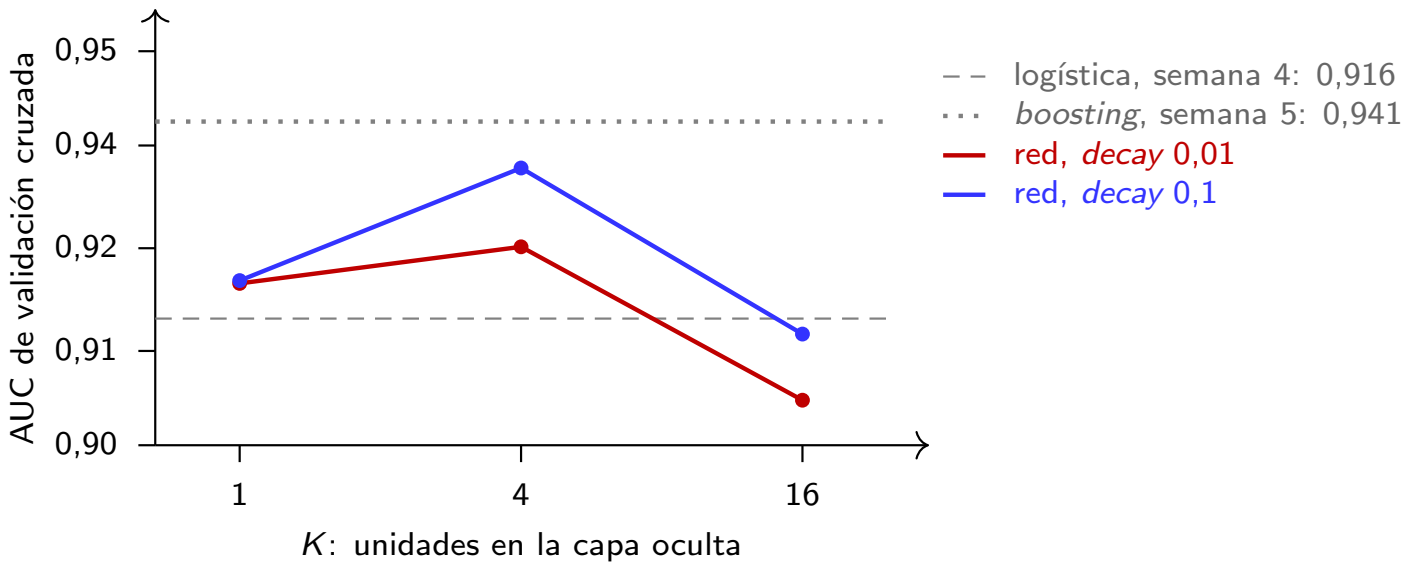
4 El ejemplo binario: ¿gana Petro el puesto?

5 Más de dos categorías: la primera vuelta

6 La carrera y el cierre

¿Cuántas unidades? La perilla K, con validación cruzada

Los diez pliegues de la semana 3; la AUC como vara, como en la semana 4



- **Cómo leerla:** cada punto es una red (censo + departamento) juzgada con los diez pliegues; el eje vertical es la AUC de validación cruzada. Rojo: *decay* 0,01; azul: *decay* 0,1 (el *weight decay* es Ridge sobre los pesos, semana 3). Las líneas grises: la logística (0,916) y el *boosting* (0,941).
- **La validación cruzada elige** $K = 4$ y *decay* 0,1, con AUC de CV 0,935: apenas 289 pesos para 9.602 puestos. **Con** $K = 16$ **la AUC cae en las dos curvas:** más unidades no siempre pagan, y la validación cruzada lo ve.

La red elegida, en la prueba

Ordenar, decidir y cuantificar: las tres lecturas de la semana 4

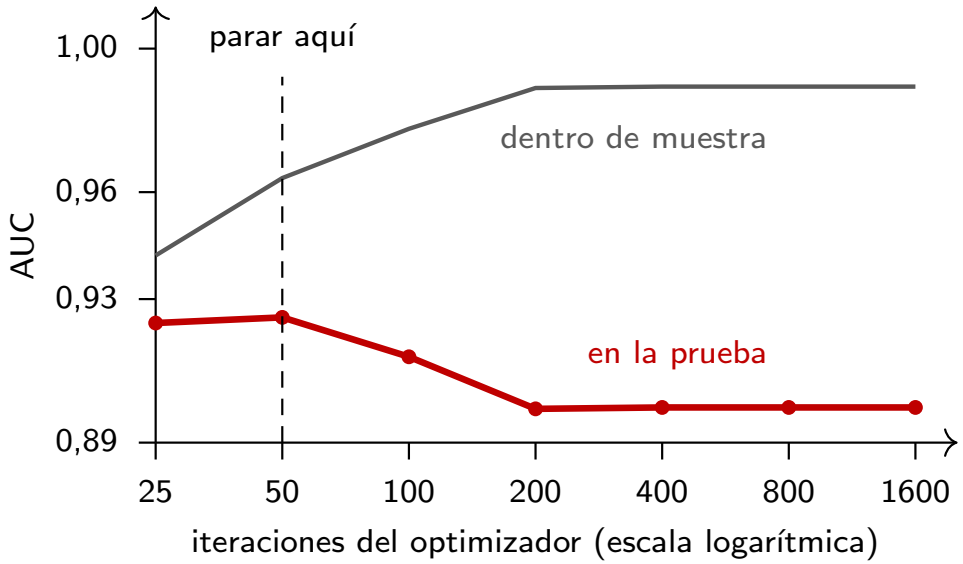
Con $\tau = 0{,}5$, la matriz de confusión en los 2.398 puestos de prueba:

	Predicho: gana Petro	Predicho: gana Hernández
Real: gana Petro	VP $= 1.095$	FN $= 173$
Real: gana Hernández	FP $= 166$	VN $= 964$

- **Ordena:** AUC de prueba 0,931. **Decide:** exactitud 85,9 % con $\tau = 0{,}5$. **Cuantifica:** Brier 0,104.
- **El umbral sigue siendo una decisión:** aquí las clases están parejas y 0,5 es razonable; con los costos de la campaña, la semana 4 lo movía a 0,83. Y si la clase fuera rara, parte 1.
- **Quedan dos perillas** que la pérdida no convexa regala: cuándo parar y dónde arrancar. Las dos láminas que siguen.

Parar a tiempo (*early stopping*): la tercera perilla

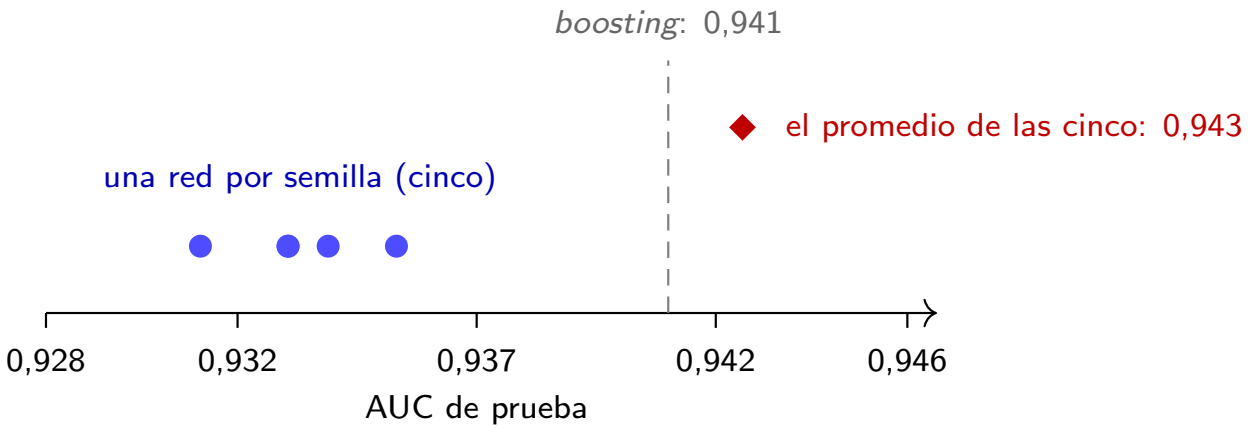
La red de 16 unidades sin *weight decay*, detenida después de 25, 50, . . . , 1.600 iteraciones



- **Cómo leerla:** la AUC dentro de muestra (gris) y en la prueba (rojo) de la misma red sin penalización, detenida en cada punto del eje. Dentro de muestra sólo sube (hasta 0,989); en la prueba toca techo en 50 iteraciones (0,925) y después baja (0,900).
- **El early stopping** es la perilla de rondas del *boosting* (semana 5): aprender de menos es un freno. En la práctica se para con un conjunto de validación, nunca con la prueba.

La semilla también es un hiperparámetro

La red elegida, entrenada con cinco semillas distintas: la AUC de cada una y la de su promedio



- **Mismos datos, misma red, otros pesos iniciales:** la AUC de prueba casi no se mueve (de 0,931 a 0,935), pero no son la misma función: como la pérdida no es convexa, cada semilla termina en otro mínimo local.
- **Promediar las probabilidades de las cinco** deja 0,943, mejor que cualquiera sola. Es el bosque otra vez (semana 5): promediar modelos que se equivocan distinto. Se reporta la dispersión entre semillas, nunca la mejor.

1 La deuda: clases desbalanceadas

2 La red neuronal, pieza por pieza

3 Cómo se aprende una red

4 El ejemplo binario: ¿gana Petro el puesto?

5 Más de dos categorías: la primera vuelta

6 La carrera y el cierre

Cuando salen varias probabilidades: el *softmax*

ISL 10.2: la misma red, con una salida por clase; las probabilidades suman uno

Cambia sólo la salida. Para M clases, la red saca M números $t_1(x), \ldots, t_M(x)$ —cada uno, su propia regresión sobre las A_k— y el **softmax** los vuelve probabilidades:

$$p_m(x) = \frac{e^{t_m(x)}}{\sum_{\ell=1}^{M} e^{t_\ell(x)}}, \qquad p_1(x) + \cdots + p_M(x) = 1.$$

- **Con $M = 2$ es la sigmoide de siempre.** La pérdida también se generaliza: la **entropía cruzada** (*cross-entropy*), $-\sum_i \log p_{m_i}(x_i)$, con m_i la clase verdadera de i (la log-loss con M clases).
- **Cuántas entran, cuántas salen:** entran las 70 columnas; salen **cuatro** probabilidades (Petro, Hernández, Fico, otros). La salida pasa de $K + 1$ pesos a $M(K + 1)$: 304 en total.
- **Se decide igual que siempre:** la clase de mayor probabilidad, o la que digan los costos. La logística también tiene esta versión (la multinomial, apéndice de la semana 4); la red le agrega las unidades.

La primera vuelta: cuatro clases, y el desbalance en acción

La red con las perillas del binario ($K = 4$, *decay* 0,1); la matriz de la prueba

Real ↓ Predicho →	Petro	Hernández	Fico	otros	Recall (%)
Petro	1.129	94	61	0	87,9
Hernández	62	606	32	0	86,6
Fico	120	43	240	0	59,6
otros	1	0	0	0	0,0

- **La referencia primero:** decir "siempre Petro" acierta el 53,8 % de la prueba; la red, el **82,7 %**.
- **El desbalance de la parte 1, en acción:** a *otros* (el 0,1 % de los puestos) la red no lo predice **nunca** (recall 0,0 %), y a Fico sólo lo encuentra 59,6 de cada 100 veces. La exactitud global lo esconde; la matriz y el recall por clase lo muestran.

Un puesto, cuatro probabilidades

El puesto más reñido de la prueba, según la red

Vereda La Sierra (Córdoba), en la primera vuelta:

	Petro	Hernández	Fico	otros
$\hat{p}$ de ganar el puesto (%)	33,5	33,0	32,9	0,6

- **Suman 100:** eso compra el softmax. En ese puesto ganó Petro.
- **Para decidir** se puede tomar la mayor… o pesar cada error con su costo, como en la semana 4: no cuesta lo mismo darle por perdido un puesto al candidato propio que al rival.
- **La misma salida sirve para** tres o diez candidatos, sectores de empleo, tipos de cultivo: cualquier y con categorías.

1 La deuda: clases desbalanceadas

2 La red neuronal, pieza por pieza

3 Cómo se aprende una red

4 El ejemplo binario: ¿gana Petro el puesto?

5 Más de dos categorías: la primera vuelta

6 La carrera y el cierre

La carrera de clasificación: todos contra todos

¿Gana Petro el puesto? La AUC de prueba de todo el curso, con la misma partición

Modelo (censo + departamento)	AUC de prueba	Semana
Decir siempre "gana Petro"	0,500	4
Regresión logística	0,916	4
k-NN en el mapa	0,945	4
Árbol	0,896	5
Bosque aleatorio	0,943	5
Boosting	0,941	5
Red neuronal	0,931	hoy
Promedio de cinco redes	0,943	hoy

- **No tenía que ganar:** hoy el punto era entender el algoritmo. La red sola queda entre la logística y los árboles; promediar las cinco semillas la sube a 0,943 y alcanza al bosque.
- **Cuándo paga la red:** con **estructura** que las columnas no traen (imágenes, texto), con **millones** de datos, o para **prestar** una representación aprendida en otra parte. En tablas como la nuestra, los ensambles de árboles mandan (Grinsztajn et al., 2022).

Recap

1. **Si una clase es rara, la exactitud engaña:** mire el reparto de la y, juzgue con la AUC y la sensibilidad/precisión de la clase rara, y decida con el umbral y los costos.
2. **Ponderar o remuestrear no crea información:** mueve el punto de decisión (lo que ya hacía el umbral) y descalibra las probabilidades; si lo hace, recalibre y dentro del pliegue.
3. **Una red neuronal es la logística con variables fabricadas:** cada unidad es una regresión doblada por g; sin g, todo colapsa a la logística.
4. **Se aprende bajando por el gradiente de la log-loss** (*backpropagation*: el residuo $y - \hat{p}$, repartido) y se regulariza con las perillas de siempre: *weight decay* (Ridge), *early stopping*, promediar semillas.
5. **Más de dos clases:** el softmax saca una probabilidad por clase y suman uno; la clase rarísima no se aprende sola.
6. **En nuestra tabla la red no destronó a los árboles:** paga con imágenes, texto o millones de datos. Entender el algoritmo era la tarea de hoy.

Preparación para la próxima clase...

Para aprovechar al máximo la próxima sesión

El martes 3 de noviembre es la sesión de los papers. Seis presentaciones de 15 minutos, una por área de la economía, con los papers elegidos en la sesión 2. Guion: la pregunta y la decisión; los datos y la unidad; la referencia; cómo se evalúa fuera de muestra y dónde falla; el resultado principal; qué método del curso es y qué habríamos hecho distinto.

Entrega 1 del proyecto, el mismo día: la idea en una página (la pregunta y la decisión que la motiva, la unidad y la y, los datos y qué se sabría en el momento de predecir), enviada antes de clase, y un *pitch* de 5 minutos. Vale 10 de los 100 puntos del proyecto.

Problem Set 2: entrega el martes 10 de noviembre a las 11:59 p. m. Con la sesión de hoy ya está todo lo que necesitan (la Parte 3 es la red contra el bosque); el 10 de noviembre no hay clase, pero sí asesoría.

Lectura para el cierre: Athey e Imbens (2019), secciones 1 a 3, para el puente causal con que termina la sesión 8.

Aplicación en R: dos scripts en `applications/week-07`: la aplicación de hoy (una red chiquita pintada por dentro —pesos, activación y la calibración a pasitos—, dos categorías de la primera vuelta y una red grande contra un *random forest*) y la réplica completa de las cifras del deck.

Referencias esenciales

- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning* (2. ed.). Springer. Cap. 10. [ISL]
- Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data? *NeurIPS Datasets and Benchmarks*.
- Dell, M. (2025). Deep Learning for Economists. *Journal of Economic Literature*, 63(1), 5–58.
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep Learning. *Nature*, 521, 436–444.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Caps. 6 a 8.
- King, G., & Zeng, L. (2001). Logistic Regression in Rare Events Data. *Political Analysis*, 9(2), 137–163.
- Athey, S., & Imbens, G. (2019). Machine Learning Methods That Economists Should Know About. *Annual Review of Economics*, 11, 685–725.
- Gelvez, J. D., Cardiles, A., Martínez-González, E. F., & Muñoz, M. (2026). How Predictable Is an Election? Working paper. [De donde salen los datos del caso.]

Apéndice: para profundizar

A1. *Backpropagation*: la derivación con una capa oculta

El modelo (regresión, pérdida cuadrática de una observación):

$$z_k = w_{k0} + \sum_j w_{kj}\, x_j, \quad A_k = g(z_k), \quad f = \beta_0 + \sum_k \beta_k A_k, \quad R = \tfrac{1}{2}\,(y - f)^2.$$

1. La capa de salida: $\dfrac{\partial R}{\partial \beta_k} = -(y - f)\, A_k$: el residuo, repartido según cuánto se activó cada unidad.

2. La capa oculta, por la regla de la cadena a través de f y de g:

$$\frac{\partial R}{\partial w_{kj}} = \frac{\partial R}{\partial f} \cdot \frac{\partial f}{\partial A_k} \cdot \frac{\partial A_k}{\partial z_k} \cdot \frac{\partial z_k}{\partial w_{kj}} = -(y - f)\, \beta_k\, g'(z_k)\, x_j.$$

3. Hacia atrás: el "error" de la unidad k es $\delta_k = -(y - f)\, \beta_k\, g'(z_k)$, y todos sus gradientes son $\delta_k\, x_j$. Con más capas, el mismo paso se repite capa por capa, de la salida hacia la entrada, reutilizando los δ ya calculados.

Con ReLU, $g'(z) = \mathbb{I}\{z > 0\}$: el error sólo fluye por las unidades activas. Con la sigmoide, $g'(z) = g(z)(1 - g(z))$, casi cero en las colas: el gradiente se desvanece.

A2. *Stochastic gradient descent*, *momentum* y Adam

Mini-batches: en vez de sumar el gradiente sobre los n datos, se usa un lote de 32 a 256 en cada paso; una **época** (*epoch*) es una pasada completa. Ruidoso pero barato, y el ruido ayuda a salir de valles malos.
Momentum: moverse en la dirección de una media móvil del gradiente,

$$m_t = \beta_1\, m_{t-1} + (1-\beta_1)\, g_t, \qquad \theta_t = \theta_{t-1} - \rho\, m_t.$$

Adam (Kingma y Ba, 2015): además escala el paso de cada parámetro por su propia volatilidad,

$$v_t = \beta_2\, v_{t-1} + (1-\beta_2)\, g_t^2, \qquad \theta_t = \theta_{t-1} - \rho\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon},$$

con $\hat{m}_t = m_t/(1-\beta_1^t)$ y $\hat{v}_t = v_t/(1-\beta_2^t)$. Los valores de siempre: $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$, $\rho \approx 10^{-3}$.

- `nnet`, el paquete de la aplicación, no usa lotes: optimiza con BFGS (un cuasi-Newton) sobre todos los datos, lo razonable para redes de cientos de pesos. Las redes de millones usan Adam.

A3. Aproximación universal: qué dice y qué no

El teorema (Cybenko, 1989; Hornik, 1991). Si g no es un polinomio (la sigmoide, ReLU), para toda función continua f sobre un compacto y todo $\varepsilon > 0$ existen K y pesos tales que

$$\sup_x \left| f(x) - \sum_{k=1}^{K} \beta_k \, g\left(w_k' x + b_k\right) \right| < \varepsilon.$$

- **Es un resultado de densidad,** como el de Weierstrass para los polinomios. No dice cuántas unidades hacen falta (K puede crecer exponencialmente con p), ni que el descenso de gradiente las encuentre, ni nada sobre el error de prueba.
- **La profundidad:** hay funciones que una red profunda representa con pocas unidades y una de una capa necesita exponencialmente muchas (Telgarsky, 2016): la composición reutiliza piezas.
- **La analogía con la semana 2:** los *splines* también aproximan cualquier función suave con nodos fijos; la red aprende *dónde* poner los nodos y en qué combinaciones de variables.

Referencias del apéndice y otras citadas

- Rumelhart, D., Hinton, G., & Williams, R. (1986). Learning Representations by Back-Propagating Errors. *Nature*, 323, 533–536. [A1]
- Kingma, D., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR*. [A2]
- Cybenko, G. (1989). Approximation by Superpositions of a Sigmoidal Function. *Mathematics of Control, Signals and Systems*, 2, 303–314. [A3]
- Hornik, K. (1991). Approximation Capabilities of Multilayer Feedforward Networks. *Neural Networks*, 4(2), 251–257. [A3]
- Telgarsky, M. (2016). Benefits of Depth in Neural Networks. *COLT*. [A3]